

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: «Алгоритм Кнута-Морриса-Пратта»

Студентка гр. 4345

Бодарева С.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Кнута-Морриса-Пратта.

Задание.

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблонов $P(|P| \leq 25000)$ и текста $T(|T| \leq 5000000)$ найдите все вхождения P в T .

Вход:

- Первая строка – P
- Вторая строка – T

Выход:

Индексы начал вхождений P в T , разделенные запятой; если P не входит в T , то вывести -1.

Sample Input:

ab
abab

Sample Output:

0, 2

Задание 2

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B . Например, *defabc* является циклическим сдвигом *abcdef*).

Вход:

- Первая строка – A
- Вторая строка – B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Sample Input:

defabc

abcdef

Sample Output:

3

Основные теоретические положения.

Алгоритм Кнута — Морриса — Пратта (КМП-алгоритм) — эффективный алгоритм, осуществляющий поиск подстроки в строке, используя то, что при возникновении несоответствия само слово содержит достаточно информации, чтобы определить, где может начаться следующее совпадение, минуя лишние проверки. Время работы алгоритма линейно зависит от объёма входных данных, то есть разработать асимптотически более эффективный алгоритм невозможно.

Префикс-функция от строки p равна массиву π , где $\pi[i]$ обозначает длину максимального префикса строки $p[0 \dots i]$, совпадающего с её суффиксом. Тривиальные случаи (префикс равен суффиксу и равен всей строке) не учитываются.

Выполнение.

Для реализации КМП необходима префикс функция. Реализована *prefix_func*, принимающая на вход строку p , создает массив pi такой же длины и заводит два счётчика $j = 0, i = 1$. Для каждой позиции i хранит $pi[i]$ — длину наибольшего собственного префикса строки $p[0 \dots i]$, который одновременно является её суффиксом. При вычислении используется $j = pi[i - 1]$, то есть длина уже найденного совпадения для предыдущего символа, сравнение $p[i]$ и $p[j]$ нужно, чтобы расширить совпадение. Если $p[i] = p[j]$, то совпадение увеличивается, т.е. $pi[i] = j + 1$. Если не совпадают, текущий префикс-суффикс всё ещё может подойти, поэтому надо перейти к следующему кандидату $j = pi[j - 1]$ — максимальной длине префикса, который может совпасть после текущего разрыва. Функция возвращает массив pi .

Алгоритм КМП реализован в функции *kmp*. На вход поступает две строки: p — образец и t — текст для поиска в нем образца. Сначала вычисляется

префикс функция для p , затем алгоритм проходится по t двумя индексами i – позиция в тексте и j – позиция в образце. Если $t[i] = p[j]$, то оба индекса увеличиваются и при $j = \text{len}(p)$ фиксируется вхождение, затем j откатывается к $p[j - 1]$, чтобы продолжился поиск следующих совпадений. Если символы не совпали и $j > 0$, алгоритм не сдвигает текст, а меняет индекс в образце $j = p[j - 1]$. Если $j = 0$ – сдвигается только индекс i . Так образец ищется в тексте без повторных проверок символов. Функция возвращает строку индексов вхождений через запятую или -1, если вхождений нет.

Для проверки, является ли строка A циклическим сдвигом B необходимо использовать КМП, взяв в качестве образца B , а в качестве текста $A \cdot 2$. Если B является подстрокой $A \cdot 2$, то A – циклический сдвиг. Для оптимизации затрат памяти строка A не удваивается, для имитации удвоения цикл проходится по строке дважды $i < 2 \cdot \text{len}(a)$, для вычисления индекса используется $i \% \text{len}(a)$.

Временная сложность алгоритма: $O(n + m)$, где n, m – длины образца и текста. Сложность алгоритма по памяти: $O(m + n)$ из-за хранения всех вхождений, если бы вхождения не сохранялись в список, то память была бы $O(m)$.

Исходный код программы см. в Приложении А.

Результаты тестирования см. в таблице 1.

Таблица 1. Результаты тестирования

№	Входные данные	Выходные данные	Комментарии
1	ab abab	0,2	Пример из задания.
2	ab vbda	-1	Образец не входит в строку
3	abab abababab	0,2,4	Пересекающиеся вхождения
4	defabc abcdef	3	Пример из задания
5	abcabc abcabc	0	Возможно несколько сдвигов, выведен первый индекс
6	abcabc abcabd	-1	A не циклический сдвиг B

Вывод

Был изучен и реализован алгоритм Кнута-Мориса-Практа для поиска вхождений образца в строку. Сложность КМП $O(n + m)$, что значительно лучше, чем $O(n \cdot m)$ для наивного алгоритма. Сложность по памяти $O(m \cdot n)$, если не хранить индексы вхождений, которых может быть n , то сложность будет $O(m)$.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: step1.py

```
def prefix_func(p: str) -> list:
    pi = [0] * len(p)
    j, i = 0, 1
    while i < len(p):
        if p[j] == p[i]:
            pi[i] = j + 1
            j += 1
            i += 1
        else:
            if j == 0:
                pi[i] = 0
                i += 1
            else:
                j = pi[j-1]
    return pi

def kmp(p: str, t: str) -> str:
    pi = prefix_func(p)
    print(f"Префикс функция от строки p: {pi}")
    solve = []
    i, j = 0, 0
    n, m = len(t), len(p)
    while i < n:
        if n - i < m - j:
            print(f"В тексте осталось: {n-i} символ(-ов), в образце - {m-j}."
Сравнение окончено")
            break
        print("-"*40)
        print(f"Сравнение {i} символа исходного текста: '{t[i]}' и {j} символа
образца: '{p[j]}'")
        if t[i] == p[j]:
            i += 1
            j += 1
            print(f"Символы совпадают")
            if j == len(p):
                solve.append(str(i-j))
                print(f"Образец найден. Индекс вхождения: {i-j}. ")
                j = pi[j-1]
            else:
                if j > 0:
                    j = pi[j-1]
                print(f"Символы не совпали, сравнение продолжится с {j} символа
образца и {i} символа исходного текста")
            else:
                i += 1
                print(f"Символы не совпали, продолжаем сравнение с {i} символа
исходного текста и {j} символа образца")
        if not solve:
            print("Образец не входит в текст.")
            return "-1"
    print("Образец входит в текст. Индекс(ы) вхождения:", end=' ')
    return ', '.join(solve)

p = input()
t = input()
```

```
print(kmp(p, t))
```

Название файла: step2.py

```
def prefix_func(b: str) -> list:
    pi = [0] * len(b)
    j, i = 0, 1
    while i < len(b):
        if b[j] == b[i]:
            pi[i] = j + 1
            j += 1
            i += 1
        else:
            if j == 0:
                pi[i] = 0
                i += 1
            else:
                j = pi[j-1]
    return pi

def shift_kmp(a: str, b: str) -> int:
    if len(a) != len(b):
        print("Длины строк не равны, одна не может быть сдвигом другой")
        return -1
    pi = prefix_func(b)
    print(f"Префикс-функция от строки B: {pi}")
    i, j = 0, 0
    n = len(a)
    while i < 2*n:
        print("-"*40)
        print(f"Сравниваем {i%n} символ строки A: '{a[i%n]}' и {j} символ строки B: '{b[j]}'")
        if a[i % n] == b[j]:
            i += 1
            j += 1
            print("Символы совпали")
            if j == len(b):
                print("Строка A - циклический сдвиг строки B. Индекс начала B в A:", end=' ')
                return (i % n)
        else:
            if j > 0:
                j = pi[j-1]
            else:
                i += 1
            print(f"Символы не совпали. Сравнение продолжится с {i%n} символа A и {j} символа B")
    print("Сравнение окончено.")
    return -1

a = input()
b = input()
print(shift_kmp(a, b))
```